

ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 5

Կառուցվածքային տիպի զանգվածների օգտագործումը \$ուկկցիաներում Պահանջ. յուրաքանչյուր խնդրի համար սահմանել համապատասխան struct, ստեղծել կառուցվածքների զանգված, իսկ հիմնական հաշվարկները, որոնումները, փոփոխությունները և դասավորումները կատարել առանձին \$ուկկցիաներով: N10. Հեծանիվների վարձակալություն Ստեղծել Bicycle կառուցվածք՝ id, type, pricePerHour, rentedHours, available դաշտերով:

Մուտքագրել 10 հեծանիվ: \$ուկկցիաների միջոցով՝

- հաշվել վարձակալված յուրաքանչյուր հեծանիվի վճարը,
- գտնել ամենաշատ ժամով վարձակալված հեծանիվը,
- արտածել տրված type-ի ազատ հեծանիվները,
- վերադարձնել տրված id-ով հեծանիվը և հաշվարկել վճարը,
- հաշվել ընթացիկ վարձակալությունների ընդհանուր գումարը:

```
#include <iostream>
```

```
#include <string>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
// Bicycle կառուցվածքի սահմանումը
```

```
struct Bicycle {
```

```
    int id;
```

```
    string type;
```

```
    double pricePerHour;
```

```
    int rentedHours;
```

```
    bool available; // true - ազատ է, false - վարձակալված է
```

```
};
```

```
// 1. $ուկկցիա՝ վարձակալված յուրաքանչյուր հեծանիվի վճարը հաշվելու համար
```

```
void calculate(const Bicycle bikes[], int size) {
```

```
    cout << "\n=== Վարձակալված հեծանիվների վճարները ===\n";
```

```
    bool foundAny = false;
```

```
    for (int i = 0; i < size; ++i) {
```

```
        if (!bikes[i].available) {
```

```
            double fee = bikes[i].pricePerHour * bikes[i].rentedHours;
```

```
            cout << "ID: " << bikes[i].id
```

```
                << " | Տեսակ: " << bikes[i].type
```

```
                << " | Վարձակալված ժամեր: " << bikes[i].rentedHours
```

```
                << " | Վճար: " << fee << " դրամ\n";
```

```
            foundAny = true;
```

```
        }
```

```
    }
```

```
    if (!foundAny) {
```

```
        cout << "Այս պահին վարձակալված հեծանիվներ չկան:\n";
```

```
    }
```

```
}
```

// 2. Ֆունկցիա՝ ամենաշատ ժամով վարձակալված հեծանիվը գտնելու համար

```
void RentedBike(const Bicycle bikes[], int size) {
    int maxIndex = -1;
    int maxHours = -1;

    for (int i = 0; i < size; ++i) {
        if (!bikes[i].available && bikes[i].rentedHours > maxHours) {
            maxHours = bikes[i].rentedHours;
            maxIndex = i;
        }
    }

    cout << "\n=== Ամենաշատ ժամով վարձակալված հեծանիվը ===\n";
    if (maxIndex != -1) {
        cout << "ID: " << bikes[maxIndex].id
            << " | Տեսակ: " << bikes[maxIndex].type
            << " | Ժամեր: " << bikes[maxIndex].rentedHours
            << " | Սակագին: " << bikes[maxIndex].pricePerHour << " դր/ժամ\n";
    } else {
        cout << "Վարձակալված հեծանիվներ չկան:\n";
    }
}
```

// 3. Ֆունկցիա՝ տրված type-ի ազատ հեծանիվները արտածելու համար

```
void printAvailableByType(const Bicycle bikes[], int size, const string& searchType) {
    cout << "\n=== Ազատ հեծանիվներ "" << searchType << "" տեսակի ===\n";
    bool found = false;
    for (int i = 0; i < size; ++i) {
        if (bikes[i].available && bikes[i].type == searchType) {
            cout << "ID: " << bikes[i].id
                << " | Ժամավճար: " << bikes[i].pricePerHour << " դրամ\n";
            found = true;
        }
    }
    if (!found) {
        cout << "Այս տեսակի ազատ հեծանիվ չի գտնվել:\n";
    }
}
```

// 4. Ֆունկցիա՝ տրված id-ով հեծանիվը վերադարձնելու և վճարը հաշվարկելու համար

```
void returnBike(Bicycle bikes[], int size, int bikeId) {
    cout << "\n=== Հեծանիվի վերադարձ (ID: " << bikeId << ") ===\n";
    for (int i = 0; i < size; ++i) {
        if (bikes[i].id == bikeId) {
            if (!bikes[i].available) {
                double totalFee = bikes[i].pricePerHour * bikes[i].rentedHours;
                bikes[i].available = true; // Հեծանիվը դառնում է ազատ
            }
        }
    }
}
```

```

        cout << "Յեժանհիվը հաջողությամբ վերադարձվեց:\n";
        cout << "Վճարման ենթակա գումարը: " << totalFee << " դրամ\n";
    } else {
        cout << "Այս ID-ով հեժանհիվը վարձակալվաժ չԷ (արդեն ազատ Է):\n";
    }
    return;
}
}
cout << "ID " << bikeId << "-ով հեժանհիվ չի գտնվել:\n";
}

```

```

double calculate(const Bicycle bikes[], int size) {
    double totalRevenue = 0;
    for (int i = 0; i < size; ++i) {
        if (!bikes[i].available) {
            totalRevenue += bikes[i].pricePerHour * bikes[i].rentedHours;
        }
    }
    return totalRevenue;
}

```

```

void AllBikes(const Bicycle bikes[], int size) {
    cout << "\n-----\n";
    cout << left << setw(8) << "ID"
        << setw(15) << "Տեսակ"
        << setw(15) << "Գին/ժամ"
        << setw(15) << "Վարձ.ժամեր"
        << setw(12) << "Կարգավիճակ" << endl;
    cout << "-----\n";
    for (int i = 0; i < size; ++i) {
        cout << left << setw(8) << bikes[i].id
            << setw(15) << bikes[i].type
            << setw(15) << bikes[i].pricePerHour
            << setw(15) << bikes[i].rentedHours
            << setw(12) << (bikes[i].available ? "Ազատ" : "Վարձակալվաժ") << endl;
    }
    cout << "-----\n";
}

```

```

int main() {
    const int SIZE = 10;
    Bicycle bikes[SIZE];

    cout << "Մուտքագրեք 10 հեժանհիվների տվյալները:\n";
    for (int i = 0; i < SIZE; ++i) {
        cout << "\nՅեժանհիվ #" << (i + 1) << endl;
        cout << "ID: ";
    }
}

```

```

    cin >> bikes[i].id;
    cout << "Տեսակ (օր. Mountain, City, Electric, BMX): ";
    cin >> bikes[i].type;
    cout << "Ժամավճար (դրամ): ";
    cin >> bikes[i].pricePerHour;
    cout << "Վարձակալված ժամեր (եթե ազատ է, մուտքագրեք 0): ";
    cin >> bikes[i].rentedHours;
    cout << "Կարգավիճակ (1 - Ազատ է, 0 - Վարձակալված է): ";
    cin >> bikes[i].available;
}

AllBikes(bikes, SIZE);

calculate(bikes, SIZE);

RentedBike(bikes, SIZE);

string targetType;
cout << "\nՄուտքագրեք փնտրվող հեծանիվի տեսակը (օր. Mountain): ";
cin >> targetType;
printAvailableByType(bikes, SIZE, targetType);

double totalRevenue = calculateTotalCurrentRevenue(bikes, SIZE);
cout << "\nԸնթացիկ վարձակալությունների ընդհանուր գումարը: " << totalRevenue
<< " դրամ\n";

int returnId;
cout << "\nՄուտքագրեք վերադարձվող հեծանիվի ID-ն: ";
cin >> returnId;
returnBike(bikes, SIZE, returnId);

cout << "\nՓոփոխված ցուցակը վերադարձից հետո:";
printAllBikes(bikes, SIZE);

return 0;
}

```